

Medical Image Interpretation (MRI, X-Ray)

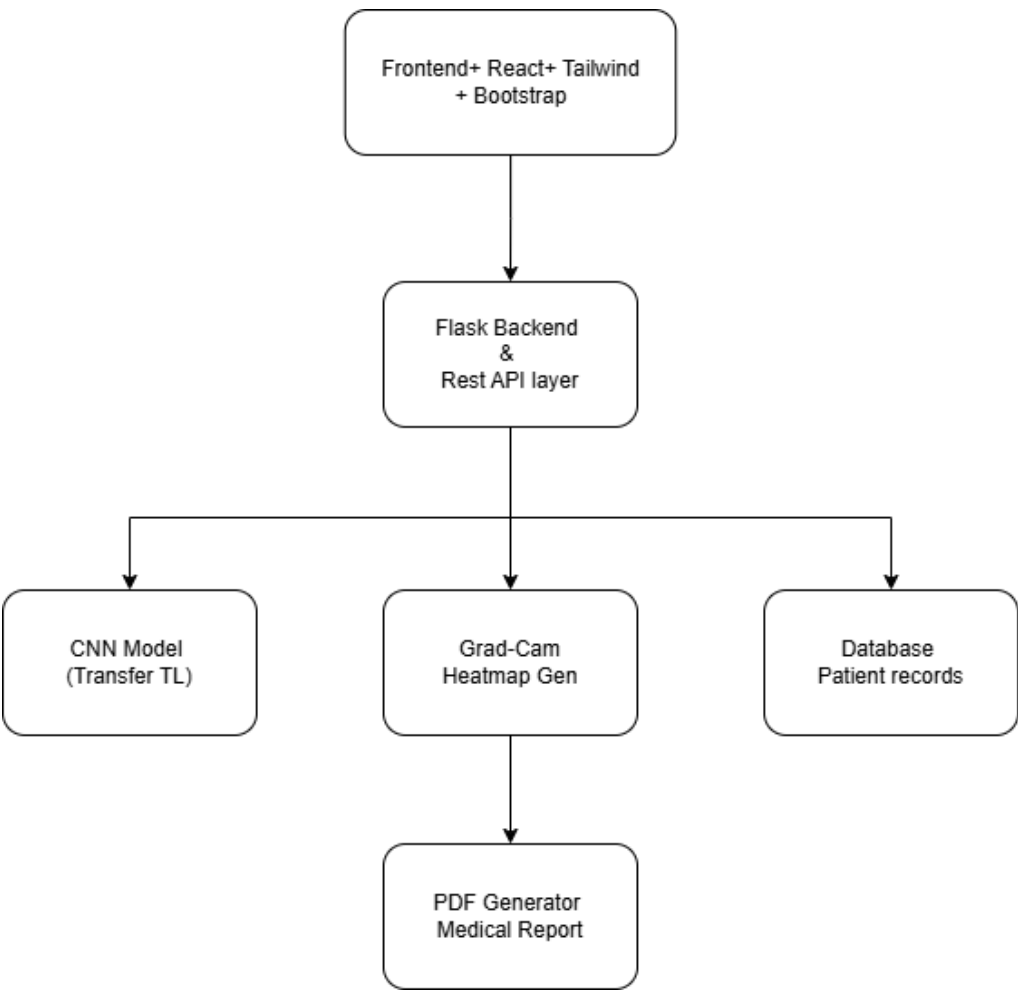
1. SYSTEM OVERVIEW

The Medical Image Interpretation system is an AI-powered web application designed to assist in the automated analysis of medical images such as MRI and X-Ray scans. The system enables users to upload medical images, receive disease predictions with confidence scores, visualize explainability through Grad-CAM heatmaps, generate downloadable PDF reports, and store patient data securely in a database.

The system follows a layered architecture with AI integration inside the backend.

2. HIGH-LEVEL ARCHITECTURE

Architecture Type: Layered Web Architecture with Integrated AI Processing



3 LAYER-WISE ARCHITECTURE

3.1 Frontend Layer

Technology:

- React
- Tailwind CSS or Bootstrap

Responsibilities:

- Allow user to upload MRI or X-Ray images
- Display image preview
- Send image to backend using REST API
- Display:
 - Predicted disease
 - Confidence score
 - Grad-CAM heatmap
- Provide PDF report download option
- Display patient history records

Communication:

- HTTP requests (POST, GET) to Flask backend
 - Data exchanged in JSON format
-

3.2 Backend Layer (Flask – Core Controller)

Technology:

- Flask (Python)

Role:

Acts as the central controller managing AI inference, heatmap generation, PDF creation, and database storage.

REST API Endpoints:

1. POST /predict
 - Accept image file
 - Perform preprocessing
 - Run CNN model
 - Generate Grad-CAM heatmap
 - Generate PDF report
 - Store results in database
 - Return JSON response

2. GET /report/<id>
 - Download generated PDF report
3. GET /history
 - Retrieve stored patient reports

4. INTERNAL BACKEND ARCHITECTURE

Project Structure:

```
app.py
├── routes.py          # API routes
├── model_loader.py    # Load pretrained CNN model
├── inference.py       # Prediction logic
├── gradcam.py         # Heatmap generation
├── pdf_generator.py   # PDF creation
├── database.py        # PostgreSQL operations
└── utils.py          # Helper functions
```

Design Principle:

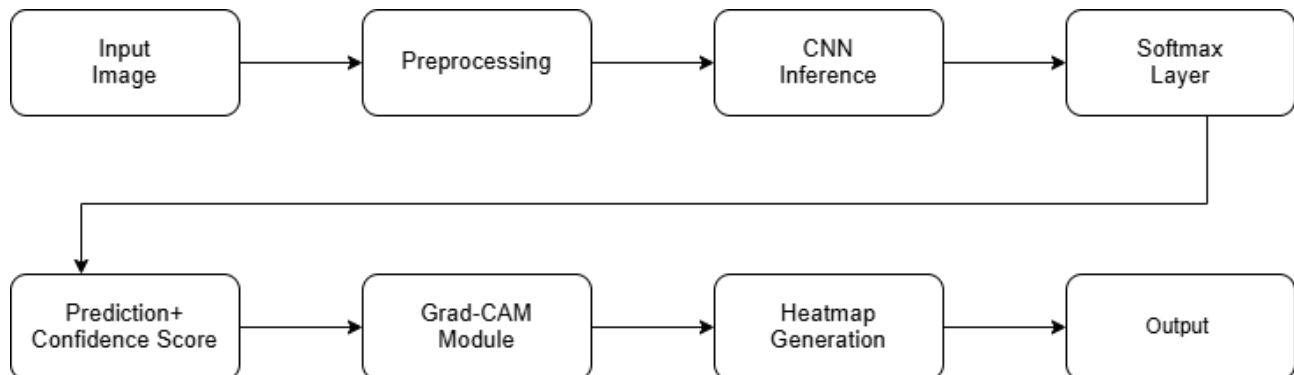
- Separation of concerns
- Modular design
- Independent AI components
- Clean database abstraction

5. AI PROCESSING ARCHITECTURE

Model Type:

- Pretrained CNN (ResNet50 / EfficientNet)
- Transfer Learning applied
- Final classification layer customized for MRI and X-Ray disease classes

AI Flow:



Preprocessing Steps:

- Resize to model input size (e.g., 224x224)
- Normalize pixel values
- Convert to tensor
- Add batch dimension

Output:

- Predicted disease label
- Confidence score
- Heatmap image

6. REPORT GENERATION LAYER

Technology:

- ReportLab (recommended)

PDF Report Includes:

- Patient name
- Uploaded MRI/X-Ray image
- Predicted disease
- Confidence score
- Grad-CAM heatmap
- Date and time
- Report ID

Reports are stored in the file system and referenced in the database.

7. DATA PERSISTENCE LAYER

Technology:

- PostgreSQL

Database Table: reports

Column Name	Type
id	SERIAL
patient_name	VARCHAR
image_path	TEXT
predicted_class	VARCHAR
confidence	FLOAT

Column Name	Type
heatmap_path	TEXT
report_path	TEXT
created_at	TIMESTAMP

Responsibilities:

- Store patient information
- Store prediction results
- Store report metadata
- Enable history retrieval

8. COMPLETE SYSTEM FLOW

1. User uploads MRI or X-Ray image.
2. React frontend sends image via POST request.
3. Flask backend receives request.
4. Image preprocessing is performed.
5. CNN model predicts disease.
6. Confidence score is calculated.
7. Grad-CAM heatmap is generated.
8. PDF report is created.
9. Data is stored in PostgreSQL.
10. JSON response is sent to frontend.
11. Frontend displays result and provides report download option.

